Inference scaling
Worked solutions

Calculations, counterexamples, and architecture decisions

This is an original problem set built around the scaling book and Reiner Pope's two Dwarkesh blackboard lessons. It assumes you have read the book. Problems and numbers are newly constructed; they are not copied exercises or Reiner's official answers.

Open this after a serious attempt. Compare assumptions and derivations before comparing final numbers. A correct result from an invalid model does not earn full credit.

Scoring

Questions 1-10:100 points. Questions 11-12:20 optional stretch points. Award points using each subpart's weight. Require equations, dimensions, feasibility, and a decision. For a calculation-heavy subpart, reserve at least one point for the reasoning, not just the number.

Review protocol

Mark each miss as tensor accounting, unit conversion, bottleneck selection, communication model, capacity, queueing, or unsupported inference. Re-solve the missed subpart without the key. End by explaining one corrected conclusion aloud in two minutes.

Coverage

Problem	Main challenge
1-3	Dense and MoE rooflines; expert occupancy; tensor-parallel communication
4-6	Cyclic decode pipelines; inverse pricing; lifetime training/serving optimization
7-8	KV transfer tails; speculative verification under load
9-10	Area and energy tradeoffs; array shape and scratchpad tiling
11-12	Training sharding; benchmark attribution and SLO goodput

Q01 Find the bottleneck before finding the batch

10 points | Conceptual source: B1, B2; R1 at 00:00:00

Tensor accounting

kappa = 2 L h_kv d s_kv = 196,608 bytes. One request at T=4,096 occupies 805,306,368 bytes. The query width is h_q d=6,144; attention FLOPs per request are aT, with a=4 L h_q d=1,179,648. Weight bytes are 60 GB. Capacity requires 60e9 + B kappa T + 8e9 <= 80e9, so B_max=14.

Times and rates

t_compute = B(60e9 + 1,179,648 T)/600e12 seconds. t_memory = (60e9 + 196,608 BT)/3e12 seconds. At T=4,096 these are 0.108053 B ms and 20 + 0.268435 B ms. For B=1,8,14: compute = 0.1081,0.8644,1.5127 ms; memory = 20.2684,22.1475,23.7581 ms. Throughput = 49.3378,361.2149,589.2728 tokens/s. All are bandwidth-bound.

A batch cannot fix every context

Compare the slopes in B: kappa T/W versus (2P+aT)/F. If F kappa > W a, the memory slope is at least the compute slope when T >= 2PW/(F kappa-Wa) = 1,573.071 tokens. At equality, memory still has the positive weight-read intercept, so no finite crossover exists. The throughput curve increases toward W/(kappa T) while on the memory branch; memory capacity truncates it earlier.

Decision

The 25 ms constraint permits B<=18 from bandwidth time, while capacity permits only 14; choose B=14. Cost = (2.40/3,600)*1e6/589.2728 = $1.13134 per million output tokens. The constraint bounds the ideal decode step, not arrivals, admission delay, prefill interference, or tail behavior.

Common failure

Using query heads instead of KV heads for cache capacity, or applying the weight-only batch crossover to long context.

Self-check: identify the assumption that would most change this answer.

Q02 MoE: active parameters are not active weight traffic

10 points | Conceptual source: B2, P1; R1 at 00:32:09

Occupancy

A particular expert is missed by one token with probability 62/64=31/32. Thus U(B)=64[1-(31/32)^B]. Expected byte traffic is [32+8U(B)] GB. Each token activates 32+16=48 GB of BF16 weights, corresponding to 24 billion parameters, so compute is 48e9 B FLOPs.

Evaluation

For B=1,8,32,128,512: U=2.0000,14.3552,40.8285,62.9003,64.0000 approximately. Compute times = 0.005,0.040,0.160,0.640,2.560 ms. Memory times = 1.0000,3.0592,7.4714,11.1500,11.3333 ms. All five are bandwidth-bound under the stipulated expectation model.

Critical batch

After all experts are effectively touched, t_memory=544e9/48e12=11.3333 ms. Set this equal to 48e9 B/9.6e15: B_crit=2,266.67, so roughly 2,267 tokens. F/W=200 FLOPs/byte and P_total/P_active=272/24=11.3333; for two-byte weights and two FLOPs/parameter, their product gives the same crossover.

Placement matters

The hotspot rank owns the hot expert and three ordinary experts. Expected routed token-expert assignments on it are B[1/4+3(7/4)/63]=B/3. The balanced mean rank receives 2B/16=B/8, so its load is 8/3 times the mean. In the expert-compute-dominated regime, use the busiest rank's compute time; pooled FLOPs alone overstates attainable throughput. Replication or adaptive placement may help but adds capacity and routing costs.

Common failure

Multiplying per-token active bytes by batch size while forgetting weight reuse, or assuming a balanced pooled compute rate despite a hotspot.

Self-check: identify the assumption that would most change this answer.

Q03 More tensor parallelism can make you slower

10 points | Conceptual source: B1, B2, P1; R1 at 00:32:09

Accounting

Local compute = B(2P+aT)/(nF). Local HBM time = [60e9/n + B kappa T/min(n,8)]/W. Add 96 t_AR. Capacity is 60e9/n + B kappa T/min(n,8) + 8e9 <=80e9. Even n=1 fits: 74.44245 GB. All listed layouts fit.

Table, in milliseconds

n=1: local 22.147484; collectives 0; total 22.147484. n=2: 11.073742; 0.542915; 11.616656. n=4: 5.536871; 1.534372; 7.071243. n=8: 2.768435; 3.470100; 6.238536. n=16: 1.518435; 7.317965; 8.836400. Local work is memory-bound throughout. Choose n=8.

Relaxed optimum

For n<=8 let A=0.022147483648 s and c=2S/W_link=1.31072e-6 s. Then t(n)=(A-96c)/n+96 alpha n+96(c-alpha). The stationary point is sqrt[(A-96c)/(96 alpha)] ~=6.773. The allowed optimum is n=8. Beyond eight ranks, KV no longer decreases and ring startup continues growing.

Replica tradeoff

One 8-way replica: 8/0.006238536 ~=1,282.35 tokens/s with 6.2385 ms service time. Four 2-way replicas: 4*8/0.011616656 ~=2,754.67 tokens/s with 11.6167 ms service time. The first favors latency; the second favors aggregate throughput when demand is sufficient. A fixed population of only eight requests cannot fill all four B=8 replicas.

Common failure

Hiding serial communication under a max, or dividing replicated KV by the full tensor-parallel degree.

Self-check: identify the assumption that would most change this answer.

Q04 A decode pipeline is a closed loop

10 points | Conceptual source: B2, P1; R1 at 00:47:12 and 01:03:37

Closed-loop schedule

For identical stages, a microbatch needs 4t to return. When M<4, each stage has M occupied slots followed by 4-M idle slots, repeating every 4t. When M>=4, stages can remain busy and each microbatch circulates every Mt. Therefore ITL=max(4,M)t and throughput=Mb/[max(4,M)t], with t in seconds. At b=150, t=3 ms: M=1 gives 12 ms and 12,500 tokens/s; M=3 gives 12 ms and 37,500; M=4 gives 12 ms and 50,000.

Optimization

Capacity gives G<=800. Since b/t_stage<=50 tokens/ms, throughput cannot exceed 50,000 tokens/s; equality requires M>=4 and b>=150. The smallest population is M=4,b=150,G=600. Its ITL is 12 ms. Other layouts can attain the same rate, but none with fewer than 600 requests under this service model.

Finite versus circulating

M/(M+3) includes draining a finite one-pass set of microbatches. A long-running closed-loop decode pipeline does not drain after every token wave. Its identical-stage steady utilization is min(M/4,1), with one initial fill transient.

Unequal stages

An isolated traversal takes 15 ms and the bottleneck has a 6 ms initiation interval, so M>=ceil(15/6)=3 is necessary. For three microbatches, start stage 1 jobs at 0,6,12,18,... ms. Each visits stage 2 three ms later, stage 3 six ms later for six ms, and stage 4 twelve ms later for three ms. Each returns after 15 ms and starts its next circuit at 18 ms. Bottleneck stays busy after fill; ITL=18 ms and throughput=150/0.006=25,000 tokens/s. Queueing raises circulation time above the 15 ms isolated traversal.

Common failure

Treating global batch size as local microbatch size, or confusing one-pass drain bubbles with an autoregressive feedback schedule.

Self-check: identify the assumption that would most change this answer.

Q05 What API prices do and do not identify

10 points | Conceptual source: B2; R1 at 01:33:02

Inversion

Cost per output token is r(m/B+kappa*T)/W. Thus p(T)=1e6*r/W*(m/B+kappa*T). The coefficient 1e6*r/W=1.25e-10. From the intercept, m/B=19.2e9 bytes and B=25. The slope is 2.40/40,000=0.000060 dollars per million per context token, giving kappa=480,000 bytes.

Counterfactual precision

Halving kappa keeps the $2.40 intercept, halves the slope to 0.000030, and changes the scale from 40,000 to 80,000. Halving m keeps the slope at 0.000060, halves the intercept to $1.20, and changes the normalized scale to 20,000. These results assume batch, bandwidth, and other costs do not change.

Non-identifiability

Both (480 GB,25,480,000 bytes) and (960 GB,50,480,000 bytes) have the same m/B and kappa. With markup mu=2, the tuple (480 GB,50,240,000 bytes) also gives the original price curve after multiplying its costs by two. Pricing alone therefore cannot identify m,B or kappa without external constraints.

Limits

Given the stipulated r,W and no-markup memory model, the curve identifies m/B and kappa; with known m it also identifies B. Real prices can include margins, routing tiers, demand-dependent batches, amortized prefill, cache hits, multiple hardware types, or compute/network cost. A byte count is not a unique architecture fingerprint.

Common failure

Presenting a model-conditioned inverse estimate as a measurement of a provider's hidden architecture.

Self-check: identify the assumption that would most change this answer.

Q06 Train longer to serve a smaller model

10 points | Conceptual source: B3; R1 at 01:18:59

Active constraint

At fixed feasible N, lowering D lowers cost until the loss constraint binds. Thus D(N)=(1-N^(-1/2))^(-2), N>1. The objective diverges as N approaches 1 from above and also as N tends to infinity, so an interior optimum exists.

Stationarity

dC/dN = 6[(1-2x)/(1-x)^3]+2R, where x=N^(-1/2). Thus R/3=(2x-1)/(1-x)^3. For R=0, x=1/2 and N=D=4. For R>0, x lies in (1/2,1); the right-hand side increases strictly from zero to infinity because its derivative is (4x-1)/(1-x)^4 >0. Hence the root is unique.

Numerical decision

For R=30: x=0.6737032969, N=2.2032442972, D=9.3923578002, C=256.3566104. Keeping N=D=4 costs 96+8R=336. The lifetime optimum reduces this artificial cost by 23.70%, while training about 2.348 times as many tokens on a smaller model.

Large-demand limit

Write epsilon=1-x. The stationarity equation is R/3=(1-2epsilon)/epsilon^3. Thus epsilon~(3/R)^(1/3), N-1~2(3/R)^(1/3), and D~(R/3)^(2/3). Lifetime serving demand can favor a smaller, more extensively trained model at fixed target loss. Real loss floors, data limits, reasoning quality, batching efficiency, latency, retraining cadence, and quantization alter the optimum; this toy law does not establish a universal overtraining factor.

Common failure

Optimizing training FLOPs alone when inference demand is part of the objective, or extrapolating a toy exponent into a factual claim.

Self-check: identify the assumption that would most change this answer.

Q07 Disaggregation: capacity is not a latency guarantee

10 points | Conceptual source: B2; original capacity and tail-latency extension

Capacity

KV bytes =196,608*32,768=6,442,450,944 bytes=6.44245 GB. Prefill capacity is 1/0.120=8.3333 requests/s per GPU, so two are needed. Decode demand is 12*500=6,000 tokens/s, so two decode GPUs suffice at 6,400 tokens/s total. Fabric demand is 77.30941 GB/s; four 25 GB/s links are needed.

Unstriped latency

Full transfer takes 257.69804 ms. Queue-free TTFT=120+257.69804+2=379.69804 ms, exceeding 250 ms even before queueing. Aggregate capacity cannot repair this individual request path.

Striping

With four-way perfect striping, a request sees 100 GB/s and transfers in 64.42451 ms. Queue-free TTFT=186.42451 ms. Fabric utilization is 77.30941/100=77.31%; prefill utilization=72%; decode utilization=93.75%. Arrival variability, synchronization, service variance, and queueing can still violate p95. Near-saturated decode capacity is particularly fragile.

Cache mixture

Expected byte fraction is 0.75*0.25+0.25*1=0.4375. Mean bytes=2.818572288 GB; mean fabric demand=33.822867456 GB/s, needing two links. An unstriped hit takes 120+64.42451+2=186.42451 ms; a miss takes 379.69804 ms. Since misses occur 25% of the time, the queue-free p95 is 379.69804 ms. Mean transfer savings do not rescue p95. Measure the actual joint distribution of prefix reuse, routing, transfer contention, and queueing under the target arrival process.

Common failure

Using fleet bandwidth to divide one unstriped request's transfer time, or replacing a tail mixture with its mean.

Self-check: identify the assumption that would most change this answer.

Q08 Speculation that helps at low load and hurts at high load

10 points | Conceptual source: B2, P2; original speculative-serving model

Expected progress

Let A be the number of accepted draft tokens, 0<=A<=K. E[A]=sum(j=1..K) Pr(A>=j)=sum(j=1..K) q^j. Each cycle also commits one correction/bonus token, so E=1+q+q^2+q^3+q^4=3.05078125. Speedup = E*t0/(d+v); effective mean service time per committed token is (d+v)/E. This is a mean, not a token-by-token latency trace.

Evaluation

At B=1: t0=8.04 ms, cycle=9.62 ms, speedup=2.54972. B=16: 8.64,11.42,2.30812. B=64: 10.56,38.62,0.83419. B=128: 13.12,75.74,0.52847. Verification consumes increasingly valuable compute as batch grows; progress alone does not determine speed.

All branches

Baseline switches at B=8/0.06=133.3333; verification switches at B=8/0.46=17.3913. Below 17.3913 the cycle is 9.5+0.12B and speculation helps for all positive B in that interval. In the middle interval the cycle is 1.5+0.58B; improvement requires B<(8E-1.5)/(0.58-0.04E)=50.017059. Above 133.3333, E*(0.10B)<1.5+0.58B for every B, so no improvement reappears. Integer answer: B=1 through 50.

Exact sampling

For draft distribution Q and target distribution P at the same accepted prefix, accept a proposed token z with probability min(1,P(z)/Q(z)). On rejection, sample from the normalized positive residual [P(z)-Q(z)]_+. If all K drafts are accepted, sample the bonus from the target at the extended prefix. Argmax agreement alone changes categorical probabilities and is not an exact stochastic sampler. Greedy decoding is a separate deterministic setting.

Common failure

Treating accepted tokens per cycle as speedup, or overlooking that verification includes more positions than the baseline.

Self-check: identify the assumption that would most change this answer.

Q09 Why four-bit arithmetic does not imply a fourfold chip win

10 points | Conceptual source: R2 at 00:00:00, 00:16:31, and 00:39:11; B1

Area and frequency

Clock rate in GHz is 1/(2/s+0.2). For p=8: s=1 gives area 160,375 lanes,0.454545 GHz,340.909 GFLOP/s; s=2 gives area 180,333 lanes,0.833333 GHz,555.000 GFLOP/s; s=4 gives area 220,272 lanes,1.428571 GHz,777.143 GFLOP/s. For p=4: s=1 gives area 96,625 lanes,0.454545 GHz,568.182 GFLOP/s; s=2 gives area 116,517 lanes,0.833333 GHz,861.667 GFLOP/s; s=4 gives area 156,384 lanes,1.428571 GHz,1,097.143 GFLOP/s.

Peak comparison

Within the allowed depths, s=4 wins for each precision. The best peak ratio is 384/272=1.411765, not four. Only one area term scales quadratically; fixed accumulation, routing, and pipeline registers consume the rest. Chip throughput depends on lane count times frequency, not frequency alone.

Workload throughput

At p=8: min(777.143,4*100)=400 GFLOP/s. At p=4: min(1,097.143,8*100)=800 GFLOP/s. Realized speedup is 2x, greater than the peak ratio because precision also changes the workload's arithmetic intensity. The claim is conditional on the stipulated halving of all byte traffic, which is often false when KV, activations, metadata, or accumulation dominate.

Energy and correctness

e(8)=86.4 pJ/MAC and e(4)=62.4 pJ/MAC. Ideal rates are 1e12/86.4=11.5741 billion MACs/joule and 1e12/62.4=16.0256 billion MACs/joule, a 1.384615x improvement. These energies do not include a pipeline-depth-dependent energy term. Validate quantization calibration, accumulation precision, overflow, and task-level quality; area arithmetic alone cannot establish numerical suitability.

Common failure

Promoting a multiplier-only area scaling law into a whole-chip throughput, energy, or model-quality guarantee.

Self-check: identify the assumption that would most change this answer.

Q10 Systolic arrays: shape, fill time, and scratchpad capacity

10 points | Conceptual source: R2 at 00:26:10 and 01:03:25; B1, B3

Large-array utilization

Useful MACs=16*1,024*256=4,194,304. There are two column tiles, each taking 1,024+128+128-2=1,278 cycles: total 2,556. Physical peak over that time is 16,384*2,556 MACs, so utilization=10.01565%. Kernel compute time is 2.556 microseconds, before HBM transfers.

Shape versus wait

The 32x128 array takes 2*(1,024+32+128-2)=2,364 cycles; utilization=43.31641%. Eight stacked requests on the large array take 2,556 cycles for 33,554,432 useful MACs, utilization=80.12520%. First request waits 140 microseconds; average batching wait is 70 microseconds. Stacking improves work per array invocation but can add far more wait than the kernel time.

Scratchpad

Occupancy=2*[2m_t K_t+2K_t n_t]+4m_t n_t=576K_t+8,192 bytes. Thus K_t<=440.8889; the largest legal multiple of 32 is 416. Partition 1024 as 416+416+192. Each column tile takes 1024+3*254=1,786 cycles; two tiles take 3,572 cycles. Re-reading X for each column tile costs 65,536 bytes; W reads 524,288 bytes; final C writes 16,384 bytes. Total=606,208 bytes. Live partial sums never spill; no initial C read is needed for C=XW.

Explicit control

A scratchpad requires explicit allocation, tile layout, buffer ownership, synchronization, and proof that an accumulator remains live across K chunks. A cache replaces some management with tags and a replacement policy, but its hit rate and interference still require analysis. Double buffering creates an opportunity to overlap movement and compute; it does not prove overlap is implemented or erase this problem's explicitly independent chunk fill/drain. A streaming design could change that timing assumption.

Common failure

Counting utilization from occupied rows alone, or silently substituting a streaming-array schedule for the given independently drained chunks.

Self-check: identify the assumption that would most change this answer.

Q11 Stretch: training memory savings can expose communication

10 points | Conceptual source: B1, B3; original full-sharding training model

Memory

Replicated persistent state uses 16*7e9=112 GB/GPU, so it fails even before activations and reserve. Full sharding uses 14 GB persistent +24 GB activations +2 GB buffers +12 GB reserve =52 GB/GPU, which fits. Sharding persistent parameter/optimizer state does not by itself shard or eliminate activations.

Time

Compute =6*7e9*8,192/(8*600e12)=71.68 ms. Two gathers plus one reduce-scatter move 3*(2*7e9)*7/8=36.75 GB/rank. Transfer time 367.5 ms, plus 96*20 microseconds=1.92 ms: total communication 369.42 ms. Serial no-overlap time 441.10 ms; ideal overlap lower bound max(71.68,369.42)=369.42 ms. Layer dependencies can make actual overlap worse than this optimistic bound.

Batch crossover versus capacity

Equating 6PS/(8F)=0.36942 s gives S=42,219.43 global tokens. If activations scale as 24 GB*S/8,192, their capacity allowance is 80-12-14-2=52 GB, so S_max=8,192*52/24=17,749.33. Integer tokens allow at most 17,749 absent additional divisibility rules. Therefore the compute/communication crossover is infeasible under the linear activation model. Checkpointing or activation sharding may change memory at additional compute/communication cost.

Common failure

Celebrating that the weights fit while ignoring communication or the activation-memory cost of the batch needed to amortize it.

Self-check: identify the assumption that would most change this answer.

Q12 Stretch: cross-examine an inference speedup claim

10 points | Conceptual source: B3, P2; original benchmark critique

What the summaries prove

A emits 6,000 tokens/s and costs 24e6/(3,600*6,000)=$1.11111 per million tokens. B emits 8,000 tokens/s and costs$0.833333 per million. A has p95 mean-ITL 35>25, so fewer than 95% satisfy that threshold; B has p95 TTFT 700>250, so fewer than 95% satisfy that threshold. Neither can meet 95% joint compliance under the stipulated exact quantiles. Exact goodput is unknown: marginal quantiles do not reveal the joint per-request distribution.

Controlled design

Match model/weights, precision, hardware count, power/configuration, prompt and output lengths, sampling, prefix reuse, cache warmness, arrival process, and measurement window. Measure per-request joint latency compliance and goodput alongside raw throughput, error rate, and quality. Include warm-up, several load points, repeated trials, uncertainty, and raw logs. Open-loop offered-load sweeps reveal queue growth that a closed-loop generator can mask by sending less work when responses slow down. Separate configuration improvements from implementation improvements.

Amdahl

Original step 20 ms; new step 15+2=17 ms. End-to-end speedup 20/17=1.17647x; latency falls 15%. Amdahl form: f=5/20=0.25, local speedup 2.5, total speedup 1/[(1-f)+f/2.5]=1/0.85. A2.5x kernel win is useful but does not establish a2.5x model-step win.

Example review

The isolated attention kernel is 2.5x faster under its reported timing conditions. The serving reports change cache-hit behavior and omit workload and arrival details, preventing attribution. Under the supplied serial breakdown, the kernel improvement predicts 1.176x model-step speedup. Run matched open-loop workloads and report end-to-end timing plus joint SLO goodput with per-request logs.

Common failure

Equating raw tokens/s with useful service, or treating marginal latency percentiles as the exact joint SLO distribution.

Self-check: identify the assumption that would most change this answer.